

DATA SCIENCE ROADMAP

A detailed beginner-to-job-ready learning plan

Python • Statistics • SQL • Data Analysis • Machine Learning • Projects • Deployment • Interviews

1	2	3	4
FOUNDATIONS	DATA SKILLS	MACHINE LEARNING	PORTFOLIO + JOB

Recommended duration: 8–12 months part-time, faster with full-time study.

How to Use This Roadmap

This roadmap is designed for a learner who wants to move from basic programming and mathematics to practical data science work. The sequence matters: build analytical foundations first, then learn machine learning, and only then add advanced topics. Do not wait until you “finish everything” before building projects—projects begin early and grow with your skills.

The 70/20/10 learning rule

Spend roughly 70% of your study time writing code and solving problems, 20% building or improving projects, and 10% watching courses or reading notes. Passive learning alone is not enough.

Roadmap at a Glance

Phase	Focus	Typical Time	Exit Outcome
Phase 0	Orientation & setup	1 week	Environment, Git, notebook workflow
Phase 1	Python foundations	4–6 weeks	Python fluency + small scripts
Phase 2	Math & statistics	4–6 weeks	Probability, inference, linear algebra basics
Phase 3	SQL & databases	3–4 weeks	Queries, joins, windows, business analysis
Phase 4	Data analysis & EDA	4–5 weeks	Pandas, visualization, cleaning, storytelling
Phase 5	Machine learning	8–10 weeks	Supervised/unsupervised ML + evaluation
Phase 6	Applied/advanced DS	4–8 weeks	Time series, NLP, experimentation, optional DL
Phase 7	Deployment & MLOps basics	3–5 weeks	APIs, Docker, cloud concepts, monitoring
Phase 8	Portfolio & interviews	Ongoing + 6 weeks	3–5 projects, resume, SQL/ML case prep

What “Job-Ready” Means

- You can take a messy dataset from raw files/database to a clean analytical dataset.
- You can use SQL to answer business questions without depending on someone else to prepare the data.
- You can explain statistical assumptions, model metrics, leakage, overfitting, bias/variance, and validation strategy.
- You can build a baseline model, improve it systematically, and explain why the improvement is credible.
- You can communicate results to both technical and non-technical audiences.
- You have public or shareable projects showing end-to-end problem solving, not only copied notebooks.
- You can discuss how a model would be deployed, monitored, retrained, and governed in production.

Phase 0 — Orientation & Development Setup

Goal: create a reliable working environment before deep study. You should be able to create a project, install dependencies, write code, track changes, and reproduce your work.

Tools to Set Up

- Python 3.x
- VS Code or another code editor
- Jupyter Notebook / JupyterLab
- Git and GitHub
- A virtual environment workflow (venv or conda)
- Command-line basics
- Optional: PostgreSQL or SQLite for local SQL practice

Core Workflow Skills

Skill	What You Should Be Able to Do
Terminal	Navigate folders, create/remove files, run Python, install packages.
Virtual environments	Create, activate, deactivate, and freeze dependencies.
Git	init, status, add, commit, branch, merge, pull, push; write useful commit messages.
Jupyter	Mix code, narrative, tables, and plots; restart and run all cells reproducibly.
Project structure	Separate data, notebooks, source code, models, reports, and README files.

Exit checkpoint

Create a GitHub repository containing a small Python program, README, requirements file, and one notebook. Clone it into a fresh folder and run it successfully.

Phase 1 — Python for Data Science

Goal: become comfortable solving problems in Python without constantly copying syntax. For data science, you need practical fluency rather than deep software-engineering mastery at the beginning.

1. Python language essentials

- Variables and primitive types
- Operators and expressions
- Strings and formatting
- Lists, tuples, sets, dictionaries
- if/elif/else
- for and while loops
- Functions, parameters, return values
- Comprehensions
- Error handling with try/except
- Reading and writing text/CSV/JSON files

2. Intermediate Python

- Modules and packages
- Scope and namespaces
- Iterators and generators—basic understanding
- lambda/map/filter only enough to read code
- Classes and objects—basic practical use
- Type hints—basic use
- Logging and debugging
- Writing reusable functions instead of notebook-only code

3. Scientific Python stack

- NumPy arrays, shapes, indexing and broadcasting
- Vectorized operations
- Pandas Series and DataFrame
- Selection, filtering, sorting
- groupby, aggregation, pivot tables
- merge/join/concat
- missing values and duplicates
- datetime handling

- apply vs vectorized operations
- Reading CSV, Excel, JSON, Parquet and database results

Practice Projects

Project	Skills Demonstrated
Expense Analyzer	File I/O, dictionaries, functions, aggregation, simple visual summaries.
Movie/Books Dataset Explorer	Pandas filtering, grouping, missing data, rankings, plots.
API-to-CSV Pipeline	Request structured data from an API, normalize fields, save clean output. If offline, use a provided JSON sample.

Do not overinvest here

You do not need advanced object-oriented design, metaclasses, decorators, or algorithm-competition techniques before starting data analysis. Learn those later if your role needs them.

Phase 2 — Mathematics & Statistics for Data Science

Goal: understand what models are doing, why evaluation works, and when a conclusion is statistically defensible. Learn mathematics through data examples rather than treating it as a separate degree.

A. Descriptive Statistics

- Mean, median, mode
- Range, variance, standard deviation
- Percentiles and quartiles
- Interquartile range and outliers
- Covariance and correlation
- Distributions, skewness and heavy tails
- Sampling and summary statistics

B. Probability

- Events and conditional probability
- Independence
- Bayes' theorem
- Random variables
- Expectation and variance
- Bernoulli, Binomial, Normal, Poisson and Exponential distributions
- Law of Large Numbers and Central Limit Theorem—intuition and practical implications

C. Statistical Inference

- Population vs sample
- Sampling distributions
- Confidence intervals
- Null and alternative hypotheses
- p-values and significance levels
- Type I and Type II errors
- Statistical power
- t-tests, chi-square tests and proportion tests
- Multiple testing awareness
- Effect size vs statistical significance

D. Linear Algebra

- Vectors and matrices
- Dot product
- Matrix multiplication
- Transpose
- Systems of linear equations—intuition
- Eigenvalues/eigenvectors—conceptual understanding
- Why linear algebra matters in regression, PCA and neural networks

E. Calculus & Optimization

- Functions and slopes
- Derivatives and partial derivatives
- Gradient intuition
- Chain rule at a conceptual level
- Loss functions
- Gradient descent and learning rate
- Convex vs non-convex optimization—basic intuition

Concept	Minimum Practical Test
Probability	Explain conditional probability with a business example.
Inference	Choose a suitable test for comparing two groups and explain assumptions.
Correlation	Explain why correlation does not imply causation and identify confounders.
Linear algebra	Explain vectors/features and dot products in a linear model.
Optimization	Explain why minimizing a loss function trains a model.

Phase 3 — SQL & Databases

Goal: become strong enough in SQL to extract, combine and analyze data independently. SQL is one of the highest-value skills for data science interviews and day-to-day work.

SQL Learning Order

Order	Topics
1	SELECT, WHERE, ORDER BY, LIMIT, aliases
2	CASE WHEN, NULL handling, string/date functions
3	GROUP BY, HAVING, aggregate functions
4	INNER/LEFT/RIGHT/FULL joins and join keys
5	Subqueries and CTEs
6	Window functions: ROW_NUMBER, RANK, LAG/LEAD, SUM/AVG OVER
7	Conditional aggregation and cohorts
8	Query performance basics: indexes, scans, explain plans—conceptual
9	Schema concepts: fact/dimension tables, normalization vs analytics models

SQL Business Questions You Should Practice

- Top products/customers by revenue
- Month-over-month growth
- Retention and churn cohorts
- Daily/weekly active users
- Conversion funnel
- Repeat purchase rate
- Average order value

- Rolling averages
- First/last event per user
- Detect duplicate or inconsistent records

Exit checkpoint

Given a relational schema with customers, orders, products and events, write a multi-CTE query with joins and at least one window function to answer a business question.

Phase 4 — Data Cleaning, EDA & Visualization

Goal: learn to understand a dataset before modeling. Strong exploratory data analysis often creates more value than jumping immediately to algorithms.

Data Cleaning Checklist

- ☐ Understand row-level grain: what does one row represent?
- ☐ Check data types and semantic types
- ☐ Measure missingness and determine whether missing values are informative
- ☐ Identify duplicates and impossible values
- ☐ Standardize categories and units
- ☐ Inspect outliers before removing them
- ☐ Validate joins for duplication or dropped records
- ☐ Create reproducible cleaning functions/pipelines
- ☐ Document assumptions and data-quality decisions

EDA Questions

- What is the target or decision variable?
- How is each important feature distributed?
- Which segments differ meaningfully?
- Are there seasonal or temporal patterns?
- Are there suspicious correlations or potential data leakage?
- Which variables have many missing values or high cardinality?
- Could collection processes create bias?
- What baseline would a simple rule achieve?

Visualization Skills

- Matplotlib fundamentals
- Bar charts and line charts
- Histograms and density views
- Box plots and violin plots—when appropriate
- Scatter plots and relationship plots
- Heatmaps for carefully selected matrices
- Small multiples/faceting
- Readable labels, units, annotations and titles
- Avoiding misleading axes and overloaded dashboards

Storytelling principle

A useful chart should answer a question. The title should communicate the finding—not merely name the chart. Example:

“Retention drops sharply after Week 2” is more useful than “Weekly Retention Chart.”

Milestone Project 1 — End-to-End EDA

Choose a real dataset and produce a professional analysis containing: problem framing, data dictionary, cleaning decisions, 10–15 meaningful analyses, 5–8 polished charts, key findings, limitations, and recommended next actions. Publish the notebook plus a concise README.

Phase 5 — Machine Learning Core

Goal: learn a disciplined machine-learning workflow. The priority is not memorizing algorithms—it is selecting an appropriate method, preventing leakage, validating correctly, and interpreting results.

Machine Learning Workflow

Step	Questions to Answer
1. Frame	What exactly are we predicting or deciding? What is the unit of prediction?
2. Baseline	What simple rule or simple model must we beat?
3. Split	How should train/validation/test data be separated? Is time ordering important?
4. Prepare	What preprocessing must be fitted only on training data?
5. Train	Which model family matches the data and objective?
6. Evaluate	Which metric reflects the real business cost?
7. Diagnose	Where does the model fail? Which segments are weak?
8. Improve	Feature engineering, tuning, class handling, more data, alternate models.
9. Interpret	Which factors drive predictions? Are explanations stable and sensible?
10. Deploy	How will predictions be served, monitored and retrained?

Supervised Learning — Regression

- Linear regression
- Regularization: Ridge/Lasso intuition
- Decision tree regression
- Random forest
- Gradient boosting methods
- Metrics: MAE, MSE/RMSE, R^2 , MAPE caveats

Supervised Learning — Classification

- Logistic regression
- Decision trees
- Random forest
- Gradient boosting
- k-nearest neighbors—conceptual/practical basics
- Support vector machines—basic intuition
- Class imbalance
- Threshold tuning
- Metrics: accuracy, precision, recall, F1, ROC-AUC, PR-AUC, log loss

Model Validation

- Train/validation/test split
- k-fold cross-validation
- Stratified cross-validation
- Time-series split

- Leakage prevention
- Hyperparameter tuning
- Learning curves and validation curves

Feature Engineering

- Scaling and normalization
- Categorical encoding
- Binning only when justified
- Date/time features
- Text-derived features
- Interactions and domain features
- Handling high-cardinality variables
- Pipelines and ColumnTransformer-style workflows

Unsupervised Learning

- k-means clustering
- Hierarchical clustering—basic understanding
- DBSCAN—conceptual use
- PCA and dimensionality reduction
- Cluster validation and the danger of inventing meaning

Critical concept: leakage

Data leakage occurs when training data contains information that would not genuinely be available at prediction time. A highly accurate leaked model can be useless in production. Always design your split before feature engineering.

Model Selection Cheat Sheet

Situation	Strong Starting Point	Why
Structured binary classification	Logistic regression + gradient boosting	Interpretable baseline plus strong nonlinear benchmark.
Structured regression	Linear/Ridge + gradient boosting	Simple benchmark plus flexible high-performing model.
Need interpretable rule structure	Shallow decision tree	Easy to inspect, though may sacrifice predictive performance.
Many nonlinear interactions	Random forest / gradient boosting	Captures interactions with limited manual transformation.
High-dimensional sparse text	Linear model on TF-IDF	Efficient and often strong baseline.
Customer segmentation	k-means after thoughtful scaling/feature design	Useful baseline if spherical cluster assumption is acceptable.
Temporal forecasting	Naive seasonal baseline + classical/ML time-series models	Forecasting requires time-aware validation and strong baselines.

Metrics: Choose by Decision Cost

Problem	Useful Metrics	Watch Out For
Balanced classification	Accuracy, F1, ROC-AUC	Accuracy can still hide segment-level failures.
Rare positive class	Precision, Recall, PR-AUC, F1	ROC-AUC can look good even with poor useful precision.
Ranking	ROC-AUC, PR-AUC, precision@k, recall@k	Threshold-free metrics may not match operating constraints.
Regression	MAE, RMSE, R^2	R^2 alone does not tell error magnitude.
Forecasting	MAE, RMSE, MASE, sMAPE	MAPE can explode when actuals are near zero.

Phase 6 — Applied & Advanced Data Science

These topics become valuable after you are comfortable with the core workflow. Choose based on the jobs you want rather than trying to master all of them.

A. Experimentation & Product Analytics

- A/B testing
- Randomization
- Sample size and power
- Primary vs guardrail metrics
- Novelty and seasonality effects
- Sequential testing awareness
- CUPED / variance reduction—later
- Causal inference basics: confounding, matching, propensity scores, difference-in-differences—optional advanced

B. Time Series

- Trend, seasonality, autocorrelation
- Lag and rolling features
- Naive and seasonal-naive baselines
- Exponential smoothing / ARIMA concepts
- ML forecasting with time-aware features
- Backtesting
- Forecast intervals
- Avoiding future leakage

C. NLP

- Tokenization and normalization
- Bag of words and TF-IDF
- Text classification baselines
- Word embeddings—conceptual
- Transformer/LLM fundamentals
- Embeddings for semantic search
- Evaluation for text tasks
- Retrieval-augmented generation concepts—optional

D. Deep Learning — Optional for General DS

- Neural network basics
- Activation functions
- Loss functions and backpropagation intuition
- Regularization and dropout
- CNN/RNN concepts if role-relevant
- Transformers if NLP/GenAI focused
- Framework exposure: PyTorch or TensorFlow; one is enough initially

Recommended priority

For a general Data Scientist role, statistics, SQL, EDA, experimentation, scikit-learn-style machine learning, and business communication usually deserve priority over deep learning. Specialize later based on target roles.

Phase 7 — Deployment & MLOps Basics

Goal: understand how analysis becomes a usable system. You do not need to become a DevOps engineer, but you should be able to package a model and explain the production lifecycle.

- Save/load models safely
- Build a simple prediction function/module
- Create a REST API with FastAPI or Flask
- Environment and dependency management
- Docker basics: image, container, Dockerfile
- Batch prediction vs real-time inference
- Cloud concepts: compute, storage, managed databases, model endpoints
- Logging and error handling
- Monitoring model inputs, outputs, latency and failures
- Data drift vs concept drift
- Model versioning
- Retraining triggers
- Basic CI/CD concepts
- Security and privacy awareness for sensitive data

Production Architecture — Mental Model

Layer	Examples of Responsibility
Data source	Database, warehouse, files, event stream, APIs.
Feature/preparation	Validation, transformations, feature computation.
Model	Trained artifact plus prediction code.
Serving	Batch job or API endpoint.
Consumer	Dashboard, application, operations team, recommendation workflow.
Monitoring	Data quality, drift, latency, errors, model performance.
Feedback	Labels/outcomes captured for evaluation and retraining.

Milestone Project 2 — Deploy a Model

Train a model on a realistic dataset, build an API that accepts input and returns a prediction, package it with Docker, write a README with architecture and run instructions, and include a model card describing data, metrics, limitations and intended use.

Phase 8 — Portfolio Strategy

A strong portfolio demonstrates judgment, not just code. Three excellent end-to-end projects are usually more convincing than ten tutorial clones.

Recommended Portfolio Mix

Project	Example Problem	Must Demonstrate
1. Analytics/EDA	Customer behavior, operations, public-policy data, sports, finance, telecom, etc.	Cleaning, SQL or data preparation, visualization, insight, business recommendations.
2. Predictive ML	Churn, fraud, default, demand, lead conversion, failure prediction.	Baseline, validation, feature pipeline, model comparison, error analysis, explainability.
3. Product/Experiment	A/B test or causal-style business question.	Hypothesis, metric design, uncertainty, statistical inference, decision recommendation.
4. Optional specialization	Forecasting, NLP, recommender, geospatial, computer vision.	Depth in a target domain.
5. Optional deployment	Serve one project with an API/app.	Reproducibility, packaging, deployment thinking, monitoring plan.

Every Project README Should Include

- ☐ Problem statement
- ☐ Why the problem matters
- ☐ Dataset and data dictionary
- ☐ Methodology
- ☐ Exploratory findings
- ☐ Baseline
- ☐ Validation design
- ☐ Final metrics
- ☐ Error/segment analysis
- ☐ Key visuals
- ☐ Business interpretation
- ☐ Limitations and risks
- ☐ How to reproduce/run the project
- ☐ Next improvements

Avoid tutorial portfolios

Do not simply publish Titanic/Iris notebooks with standard copied code. If you use a common dataset, add a distinct question, stronger validation, deep error analysis, deployment, or a domain-specific decision layer.

9-Month Part-Time Study Plan

Assumption: approximately 10–15 focused hours per week. Adjust the pace based on prior programming and mathematics experience.

Month	Main Focus	Deliverable
Month 1	Python fundamentals + Git	Daily Python problems; 1 small script project.
Month 2	NumPy/Pandas + descriptive statistics	Clean and analyze 2–3 datasets.
Month 3	Probability/inference + SQL	SQL drills; statistical analysis mini-project.
Month 4	EDA + visualization	Milestone EDA project; presentation-quality charts.
Month 5	Regression + classification	Build baselines and first ML pipelines.
Month 6	Trees, ensembles, validation, tuning	Predictive ML milestone project.
Month 7	Experimentation + one specialization	A/B test project + time series/NLP depending on target.
Month 8	Deployment + MLOps basics	API/Docker deployment project; improve GitHub READMEs.
Month 9	Portfolio polish + interviews + applications	SQL practice, ML theory, case studies, resume, mock interviews.

Weekly Routine Example

Day/Block	Activity
3 × 60–90 min	Concept learning + coding exercises.
2 × 60 min	SQL/statistics drills.
1 × 2–3 hr	Project building.
1 × 60 min	Review notes, refactor code, update README.
Every 2 weeks	Explain one concept aloud or write a short technical article.

Choose the Roadmap for Your Background

Starting Point	What to Compress	What to Emphasize
Complete beginner	Nothing initially; follow phases in order.	Python, SQL, statistics, consistent practice.
Software engineer	Python syntax and Git can be compressed.	Statistics, experimentation, EDA, business framing, ML validation.
Analyst / BI professional	Basic SQL/visualization can be compressed.	Python, statistics depth, ML, software/reproducibility practices.
Math/statistics background	Probability/inference theory may be compressed.	Python/Pandas, SQL, production workflows, portfolio projects.
ML engineer aspirant	Basic EDA can be shorter after competence is shown.	Software design, ML systems, deployment, pipelines, cloud, monitoring.

Role Differences

Role	Primary Emphasis
Data Analyst	SQL, dashboards, descriptive statistics, business metrics, communication.
Data Scientist	Statistics, SQL, EDA, experimentation, ML, business decision-making.
Product Data Scientist	SQL, experimentation, causal thinking, product metrics, stakeholder communication.
ML Engineer	Software engineering, model serving, pipelines, cloud, scaling, observability.
Applied Scientist	Deeper modeling, experimentation, domain research, advanced math/ML.
Analytics Engineer	SQL, data modeling, transformation pipelines, warehouse quality and reliability.

Data Science Interview Preparation

1. SQL

- Joins and duplicate behavior
- GROUP BY/HAVING
- CTEs and subqueries
- Window functions
- Date arithmetic
- Funnels, cohorts and retention
- Top-N per group
- Running totals and rolling metrics

2. Statistics

- Conditional probability and Bayes
- Confidence intervals
- Hypothesis tests
- p-values and power
- Sampling bias
- A/B tests
- Correlation vs causation
- Regression assumptions
- Common distributions

3. Machine Learning

- Bias vs variance
- Overfitting and regularization
- Cross-validation

- Leakage
- Metrics and threshold trade-offs
- Class imbalance
- Feature engineering
- Tree models vs linear models
- Calibration
- Interpretability
- Data drift and monitoring

4. Product / Business Cases

- Clarify objective and users
- Define success metric and guardrails
- Identify available data
- Propose analysis or experiment
- Discuss confounders and data quality
- Translate output into a decision
- State risks and limitations

5. Project Discussion

- Explain the problem in 60 seconds
- Justify dataset and target
- Explain your baseline
- Defend your split/validation
- Describe one failed idea and what you learned
- Discuss error analysis
- Explain deployment and monitoring
- Describe what you would do with more time/data

Interview rule

Do not answer only with a model name. Start with the problem, data, evaluation design, baseline, constraints and business cost; the algorithm comes after those decisions.

Learning Resource Strategy

Use one primary resource per topic, not five. Finish enough of it to build something, then switch to practice. Resource brands change over time; the categories below stay useful.

Topic	Best Resource Type	Practice Method
Python	Structured beginner course + official docs	Small scripts and 20–30 coding exercises.
Pandas/NumPy	Hands-on notebook course/documentation	Recreate analyses from raw CSV files.
Statistics	Intro statistics textbook/course with exercises	Simulate distributions/tests in Python.
SQL	Interactive SQL practice platform	Solve business-style queries every week.
ML	Applied ML course + reference textbook	Implement full pipelines and compare baselines.
Visualization	Visualization principles + plotting docs	Redesign bad charts and narrate findings.
MLOps	Practical deployment tutorials	Serve one model end-to-end.
Interviews	Question banks + mock interviews	Timed SQL/statistics/ML case practice.

Suggested Core Libraries to Know

- NumPy — numerical arrays and vectorized operations
- Pandas — tabular data manipulation

- Matplotlib — foundational plotting
- scikit-learn — preprocessing, models, pipelines, metrics
- SciPy — scientific/statistical utilities
- statsmodels — statistical modeling and inference
- Jupyter — interactive analysis
- FastAPI or Flask — lightweight model APIs
- Optional by specialization: PyTorch/TensorFlow, XGBoost/LightGBM/CatBoost, time-series/NLP/geospatial libraries

Common Mistakes That Slow Learners Down

Mistake	Better Approach
Watching courses continuously	Code after every concept and create small deliverables.
Learning every algorithm	Master the workflow and a smaller set of strong baseline/model families.
Ignoring SQL	Treat SQL as a core data-science skill, not a side topic.
Starting deep learning too early	First become strong in statistics, structured ML and evaluation.
Focusing only on model accuracy	Connect metrics to business cost and perform error analysis.
Using test data repeatedly	Keep a clean test set for final evaluation.
Feature engineering before split	Split first; fit transformations on training data only.
Copying notebooks	Rebuild from scratch and explain each decision.
Projects without a README	Write the project like a short professional case study.
Avoiding communication practice	Explain findings in plain language and make concise recommendations.

Job-Ready Checklist

Programming & Data

- ☐ Write clean Python functions
- ☐ Use NumPy/Pandas fluently
- ☐ Load, clean, reshape and join datasets
- ☐ Use Git for project history

SQL

- ☐ Write multi-table joins
- ☐ Use CTEs and window functions
- ☐ Build cohorts/funnels/retention queries
- ☐ Reason about duplicates and grain

Statistics

- ☐ Explain confidence intervals and p-values
- ☐ Choose basic hypothesis tests
- ☐ Understand power and Type I/II errors
- ☐ Explain confounding and causality limits

Machine Learning

- ☐ Build regression and classification pipelines
- ☐ Use correct validation

- ☐ Select appropriate metrics
- ☐ Avoid leakage
- ☐ Perform feature engineering and error analysis
- ☐ Explain model trade-offs

Portfolio

- ☐ At least 3 strong end-to-end projects
- ☐ At least 1 project with deployment or production thinking
- ☐ Clear GitHub READMEs
- ☐ Projects show business decisions, not only notebooks

Communication & Interviews

- ☐ Give a concise project walkthrough
- ☐ Solve SQL under time pressure
- ☐ Explain ML concepts without jargon
- ☐ Structure open-ended case studies
- ☐ State assumptions and limitations clearly

You are ready to start applying before every box is checked

A practical threshold is: strong Python + SQL + EDA, solid statistics fundamentals, core supervised ML, and 2–3 credible projects. Continue learning while applying; job descriptions often list more technologies than any one person uses daily.

Final Recommended Sequence

Python → NumPy/Pandas → Statistics → SQL → EDA/Visualization → Machine Learning → Experimentation/Specialization → Deployment → Portfolio → Interviews